

Chapter 3: Introduction to Web Applications

Gregory S. DeLozier, PhD

Our Python database script carried out a sequence of operations and stopped. A web application waits for requests. Someone opens a page, submits a form, or follows a link, and the application decides what work that request requires.

The example in `chapter-03-intro-to-web-apps/code` puts a browser interface on the pet database. Python still issues Structured Query Language (SQL) statements through `sqlite3`. What changes is how we choose an operation and where its inputs come from. The application reads a request, works with stored state, and sends back a response.

This chapter follows `app.py`, the files in `templates`, and the address examples in `example.txt`. Code excerpts retain the example's behavior, with formatting adjusted for readability. The examples follow a pet from the browser form to the database and back to the displayed list.

Running the Application

Work from the `chapter-03-intro-to-web-apps/code` directory. It contains `app.py`, the supplied `pets.db`, and the `templates` folder. Make a working copy of this directory for practice; creating, editing, and deleting pets changes its database.

For a new Python environment on macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install flask
flask run
```

Flask discovers the application in `app.py`. The explicit equivalent is `flask --app app run`. The terminal shows the address to open, normally:

```
http://127.0.0.1:5000/
```

Open `/pets` at that address to see the database interface. Leave the terminal running while using the application. Stop the server with Control-C. If port 5000 is already occupied, use `flask run --port 5001` and open the address it reports. This is Flask's development server, intended for local development.

The application uses the existing pet table when it starts. Run it from the topic directory so the relative filename `pets.db` refers to the supplied database.

What an Address Tells Us

A Uniform Resource Locator (URL) identifies the resource a browser is requesting. The topic's example `le.txt` takes an address apart. Here is a similar example with the parts together:

```
https://mywebsite.com:8443/catalog?category=blenders&brand=KitchenAid
```

Part	Meaning
https	Scheme: use encrypted Hypertext Transfer Protocol (HTTP)
mywebsite.com	Hostname
8443	Explicit port number in this example
/catalog	Path requested from the server
category=blenders&brand=KitchenAid	Query string containing named values

The Domain Name System (DNS) resolves a hostname to an Internet Protocol (IP) address. A port helps direct the connection to a service at that address. When omitted, the usual port is 443 for HTTPS and 80 for HTTP. Our local Flask server uses HTTP on port 5000 instead.

A path is not necessarily a file on the server. `/pets` selects application code that builds a page. There is no requirement for a file named `pets` to exist.

The notes also contrast values in a path with values in a query string:

```
/catalog/blenders  
/catalog?category=blenders
```

These discussion examples show two ways to supply a category: as part of the path or as a named query parameter. The route function reads the value and decides how to use it. Our pet application uses a value in the path to identify the record being edited.

Requests, Routes, and Responses

A browser sends an HTTP request containing a method and a target address. A GET requests a representation, such as a page or an edit form. In this application, POST submits form values for creating or updating a record.

Flask matches the request to a route and calls its Python function. The function's return value becomes a response. A response includes a status code and may include a body, such as Hypertext Markup Language (HTML), or a redirect telling the browser to request another address.

The application starts with:

```
from flask import Flask, render_template, request, redirect, url_for

import sqlite3
from pprint import pprint

connection = sqlite3.connect("pets.db", check_same_thread=False)
print("succeeded in making connection.")

app = Flask(__name__)
```

app is the Flask application object. The connection is an in-memory Python object through which the application accesses the database file. pprint prints debugging output in the server terminal; it does not put anything on the browser page.

The simplest route is:

```
@app.route("/bye", methods=["GET"])
def get_bye():
    return "Bye!"
```

The decorator registers the function for GET /bye. Defining the function does not send a page to anyone. Flask calls it when a matching request arrives.

Here are the main operations in the pet interface:

Request	Function	Work
GET /pets	get_pets	Load records and display the table
GET /create	get_create	Display a blank form
POST /create	post_create	Insert a pet, commit, and redirect
GET /update/<id>	get_update	Load a pet and fill an edit form
POST /update/<id>	post_update	Save submitted changes and redirect
GET /delete/<id>	get_delete	Delete a pet and redirect

Passing Values to a Template

The greeting function has three routes:

```
@app.route("/", methods=["GET"])
@app.route("/hello", methods=["GET"])
@app.route("/hello/<name>", methods=["GET"])
def get_hello(name="world"):
    return render_template("hello.html", name=name)
```

/ and /hello use the default name, world. A request for /hello/Ada supplies the string "Ada" as the function argument. The <name> part is a variable in the route, not text the user types literally.

The template hello.html contains:

```
<html>
  <body>Hello, {{ name }}! (from the template)</body>
</html>
```

render_template evaluates this file using Jinja, Flask's template engine. The keyword argument name=name makes the Python value available under the template name name. The browser receives HTML containing the completed greeting. It does not run the Python function or evaluate the Jinja expression.

The double braces insert a value. In these HTML templates, Flask enables automatic escaping, so characters such as < in an ordinary supplied string are displayed as text rather than interpreted as an HTML tag.

This is our first separation of responsibilities: the route decides what data to provide, and the template describes the resulting page.

Loading and Displaying Pets

The list route reads the database before rendering its page:

```
@app.route("/pets", methods=["GET"])
def get_pets():
    cursor = connection.execute("select * from pet")
    rows = cursor.fetchall()
    pprint(rows)
    return render_template("pets.html", pets=rows)
```

The supplied table has columns id, name, kind, age, and food, in that order. Each result row is a tuple. rows is a list of those tuples, passed to the template as pets.

Inside the table in `pets.html`, the template uses nested loops:

```
{% for pet in pets %}
<tr>
  {% for item in pet %}
    <td>{{ item }}</td>
  {% endfor %}
  <td><a href="/update/{{pet[0]}}">
    <span class="material-symbols-outlined">edit</span>
  </a></td>
  <td><a href="/delete/{{pet[0]}}">
    <span class="material-symbols-outlined">delete</span>
  </a></td>
</tr>
{% endfor %}
```

`{% ... %}` marks template statements, including loops. The outer loop creates a table row for each pet. The inner loop creates a cell for each column value.

`pet[0]` is the record's identifier. It becomes part of the edit and delete links. For a pet whose identifier is 3, the edit link points to `/update/3`. A name might change or be shared by several pets; the identifier selects the stored record.

The table's appearance uses Bootstrap styles and externally loaded icon fonts. Those resources affect presentation. The database query and template loops still determine the actual records and links. The current route renders `pets.html`.

Opening and Submitting a Form

The create page begins with a GET:

```
@app.route("/create", methods=["GET"])
def get_create():
    return render_template("create.html")
```

The form in that template is:

```
<form action="/create" method="post">
  <p>Pet Name:<input name="name"/></p>
  <p>Kind:<input name="kind"/></p>
  <p>Age:<input name="age"/></p>
  <p>Food:<input name="food"/></p>
  <button type="submit">Create</button>
  <a href="/pets">Cancel</a>
</form>
```

The action specifies where to submit the form. The method specifies how. Each input's name becomes the key associated with its submitted value. The text `Pet Name:` is a label for the reader; the application receives the key name because of the `input` attribute.

Clicking Create sends a separate `POST /create` request with the form values in the request body. That request performs the insertion. Cancel is an ordinary link back to `/pets` and leaves the database as it was.

Creating and Saving a Pet

The POST route handles the submitted values:

```
@app.route("/create", methods=["POST"])
def post_create():
    data = dict(request.form)
    connection.execute(
        """insert into pet(name, kind, age, food)
           values (?, ?, ?, ?)""",
        (data["name"], data["kind"], data["age"], data["food"])
    )
    connection.commit()
    return redirect(url_for("get_pets"))
```

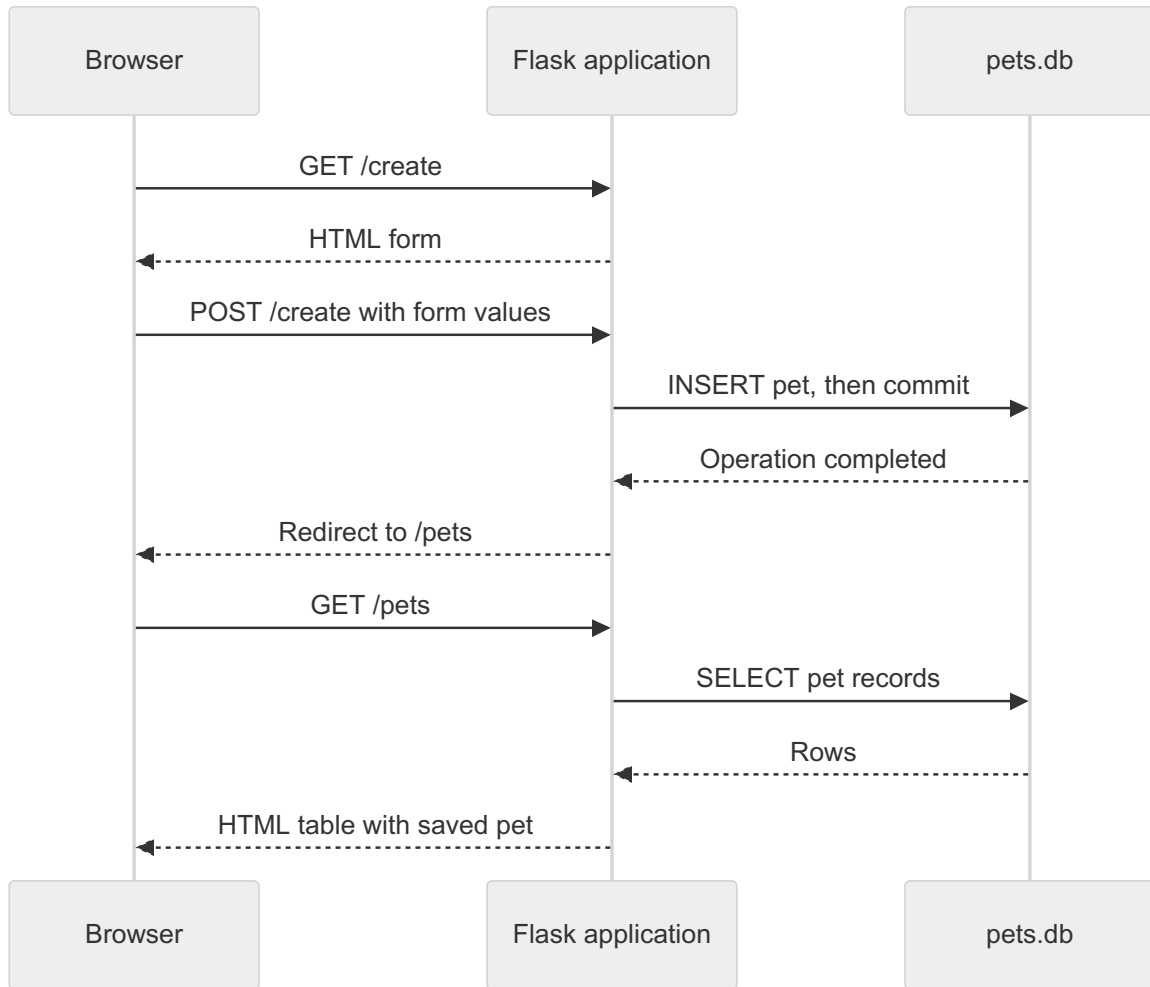
`request.form` contains parsed form data for this request. Converting it to a dictionary gives the function a familiar way to retrieve the fields. These input values arrive as strings, including age.

The SQL statement leaves out `id`; SQLite supplies the identifier for the new record. The question marks bind the submitted values as data. A pet named `O'Malley` does not require the application to build an SQL string with an escaped apostrophe.

`execute()` performs the insertion. `commit()` completes the transaction so the change is saved.

Finally, `url_for("get_pets")` finds the URL associated with that function name. `redirect()` returns a redirect response, normally with status 302. The browser follows it with GET `/pets`, which queries the database again and renders the updated list.

A Create Request, Followed by a Fresh Display



The form submission and the final display are separate requests. Refreshing the final list requests that list again, rather than ordinarily resubmitting the insertion. This pattern is often called Post/Redirect/Get.

Loading a Record for Editing

Editing begins by loading the selected pet. The route `/update/<id>` supplies an identifier as text; `get_update` converts it with `int(id)` before querying:

```
cursor = connection.cursor()
cursor.execute("""select * from pet where id = ?""", (id,))
rows = cursor.fetchall()
```

The identifier is supplied through the SQL parameter, `(id,)`, whose trailing comma makes it a one-element tuple.

For a matching record, the function unpacks the row and constructs a dictionary:

```
(id, name, kind, age, food) = rows[0]
data = {
    "id": id,
    "name": name,
    "kind": kind,
    "age": age,
    "food": food
}
```

It then returns `render_template("update.html", data=data)`. The dictionary gives the template named fields for the selected pet.

The edit template uses the values to fill the form. These lines are excerpts:

```
<form action="/update/{{data['id']}}" method="post">
    <p>Pet Name:<input name="name" value="{{data['name']}}"/></p>
    <p>Kind:<input name="kind" value="{{data['kind']}}"/></p>
    <p>Age:<input name="age" value="{{data['age']}}"/></p>
    <p>Food:<input name="food" value="{{data['food']}}"/></p>
```

For record 3, the completed form submits to `/update/3`. The `value` attributes fill the inputs with the current name, kind, age, and food. The person edits these starting values and submits the form to save the changes.

Submitting an Edit

The save operation uses POST `/update/<id>`. It obtains the submitted fields and converts age:

```
data = dict(request.form)
try:
    data["age"] = int(data["age"])
except:
    data["age"] = 0
```

`int()` converts the submitted age from text to an integer. The exception branch supplies zero when conversion fails.

The update itself is parameterized:

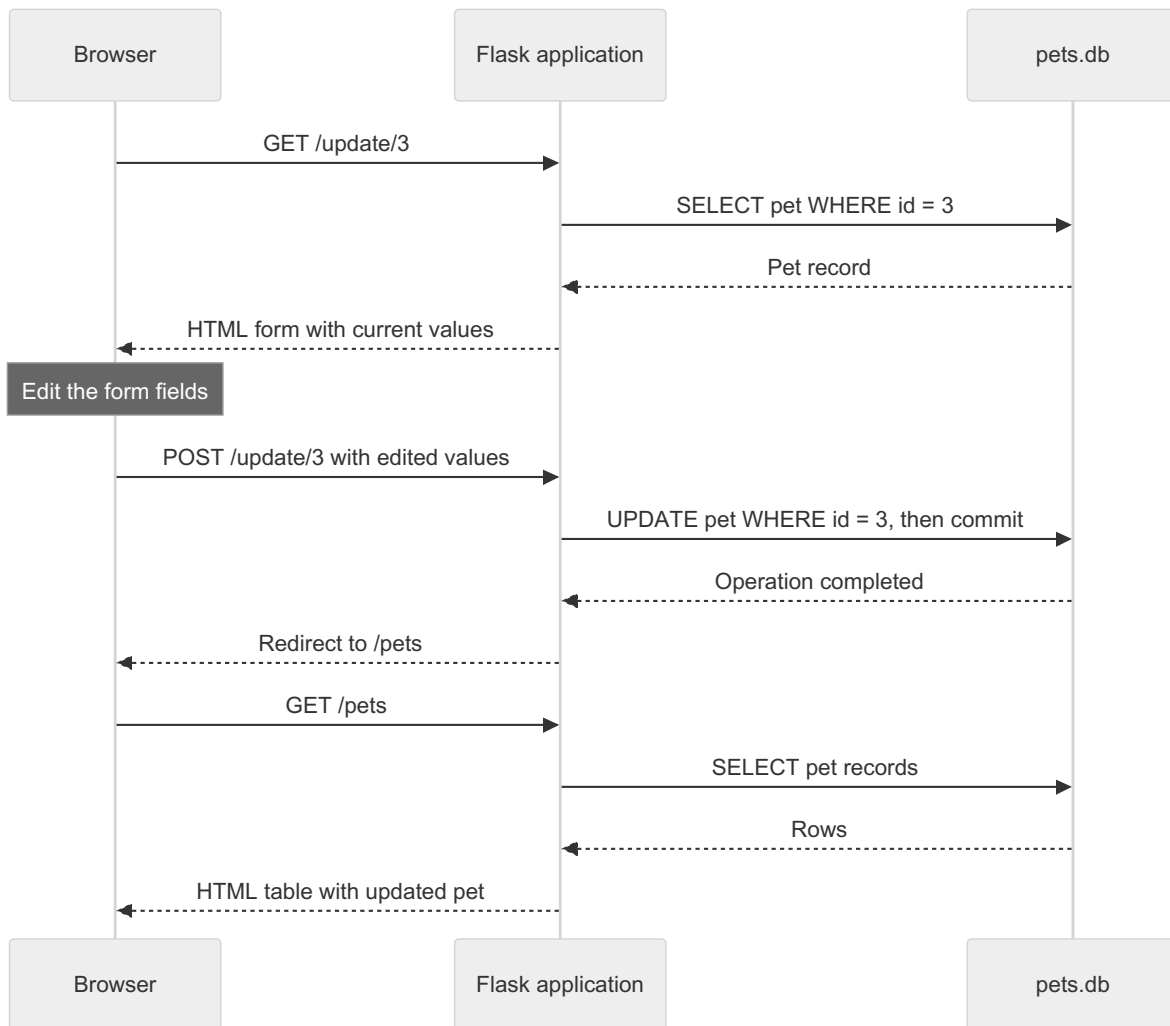
```
cursor = connection.cursor()
cursor.execute(
    """update pet set name=?, kind=?, age=?, food=? where id=?""",
    (data["name"], data["kind"], data["age"], data["food"], id)
)
connection.commit()
return redirect(url_for("get_pets"))
```

The identifier chooses the row; the other parameters supply replacement values. After committing, the redirect leads to a fresh query and display.

This is the persistence cycle spread across browser requests. The edit link determines the action and selects a record. The first request loads its state. The person changes the form, and a later request saves those changes. Display happens both when the form is filled and when the updated list is shown.

The identifier in the form action connects the submitted edit to the selected record. Both requests use that identifier: the first to load the pet, the second to save its new values.

Loading, Editing, and Saving a Pet



Here, 3 stands for the identifier of the pet selected from the list. The first database read fills the edit form. After saving, the second read supplies the updated list page.

Deleting and Handling Missing Records

Deletion is short:

```
@app.route("/delete/<id>", methods=["GET"])
def get_delete(id):
    id = int(id)
    cursor = connection.execute(
        """delete from pet where id==?""", (id,)
    )
    connection.commit()
    return redirect(url_for("get_pets"))
```

The delete link includes the pet's identifier in its address. Following `/delete/3` calls this function with `id="3"`. The function converts the identifier, deletes that row, commits, and redirects to `/pets`. SQLite accepts `==` as an equality operator in the statement.

The edit route also uses `error.html` to display a message when it cannot load a pet. A request to `/update` displays `No ID was provided`. A numeric identifier with no matching row produces `Data not found`. The template places the supplied message on the page and provides a link back to `/pets`.

Following a Request

For now, trace one operation all the way through. Identify the request, find its route, follow the database work, and inspect the response. That makes it much easier to tell whether a problem is in the form, the Python code, or the stored data. The database-abstraction topic will give us a place to collect database operations instead of repeating them throughout route functions.

Practice

Use a working copy of the topic directory and its database.

1. Start the application with `flask run`. Visit `/`, `/hello/Ada`, `/bye`, and `/pets`. Match each response to its route function and template, if any.
2. Create a pet named O'Malley. Find the record using the SQLite shell. Compare the submitted form fields with the stored columns, including the generated identifier.
3. Open the browser's developer tools and inspect network requests while creating a pet. Find the form submission, redirect, and subsequent list request. Record their methods and status codes.
4. Open an edit form and change a field without submitting it. Query the database from the shell. Then submit the form and query again. Explain when the stored value changes.
5. Change the list query to display pets in name order. Keep the template unchanged and verify the result.
6. Follow an edit in the browser's network tools. Identify the request that loads the form and the request that submits it. Compare them with the update-flow diagram.
7. Create a practice pet, note its identifier, then delete it using the list page. Follow the request and redirect, and confirm the row is gone using the SQLite shell.

8. Create a pet, stop the server, and start it again. Confirm that the record remains. Explain why this differs from restarting the earlier script that rebuilt its table.

Questions for Thought and Study

1. Why can `/pets` work when there is no file named `pets`?
2. How can `GET /create` and `POST /create` call different functions?
3. Which parts of `hello.html` are evaluated by Jinja, and what does the browser receive?
4. What connects a submitted form field to `data["name"]`?
5. Why does an input named `age` arrive as a string?
6. What is the difference between a record identifier and its position in the displayed table?
7. Why does changing the edit form not immediately change the database?
8. What does the redirect accomplish after a successful insert?
9. How does the identifier get from a row in the pet list to the update or delete function?
10. Which state survives a server restart, and which objects must be created again?

Further Reading and Practice

- **Web applications:** Background on applications accessed through a browser.
https://en.wikipedia.org/wiki/Web_application
- **Flask quickstart:** Running the server, routes, templates, and request data.
<https://flask.palletsprojects.com/en/stable/quickstart/>
- **Flask database tutorial:** Managing a database connection during a request.
<https://flask.palletsprojects.com/en/stable/tutorial/database/>
- **Sending form data:** How form actions, methods, and named controls determine a request.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Sending_and_retrieving_form_data